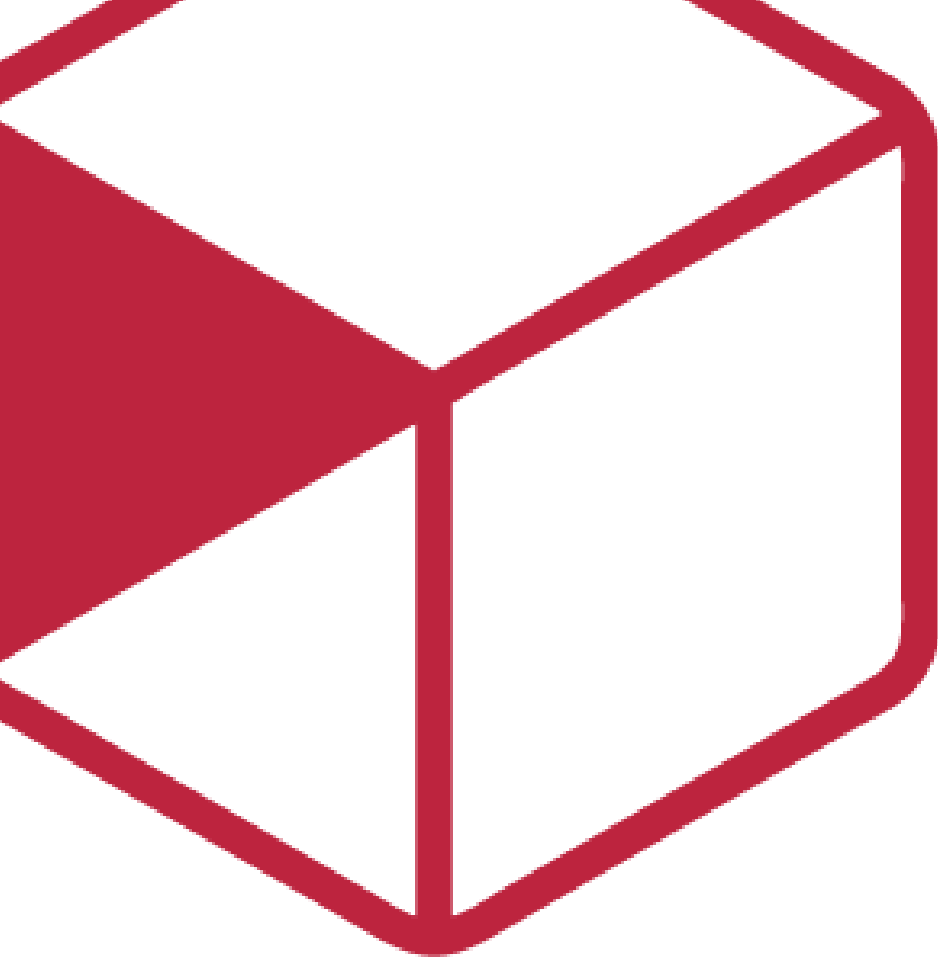




Diseño de la solución

R16A-RE-FU-006

FORMATO	Arquitectura
PROYECTO	R16 - Adquisiciones
REFERENCIA	AUI- FOR-01
VERSIÓN	1.1
FECHA	1 jul 2026
AUTOR	Jose Armando Santiago Lorenzo
REVISOR	Juan David García Cruz



Contenido

Contenido	2
Importante	3
Introducción	4
1. Propósito del documento	4
2. Alcance	4
Específicamente incluye:	4
No se consideran:	4
Visión general del diseño	6
1. Objetivo técnico	6
2. Componentes involucrados	6
Diseño funcional detallado	9
1. Flujo 1 — Asignación de cuenta bancaria al cliente (CREATE / UPDATE)	9
2. Flujo 2 — Asignación de referencia bancaria al generar proforma	10
Algoritmo Banamex — 7 segmentos (invocado en Flujo 1)	10
3. Criterios de aceptación del requisito	12
4. Reglas técnicas aplicadas	14
Diseño de componentes	16
1. Diagramas	16
Diseño de Interfaces	18
1. Interfaces de entrada	18
2. Interfaces de salida	19
3. Contrato de datos	19
4. Tabla de EndPoints	19
Diseño de Modelo de Datos	21
1. Nueva tabla — dbo.ClienteDatosBancarios	21
2. Modificaciones en BD existente	22
Impacto Técnico	24
1. Impacto en código existente	24
⚠ Corrección requerida en ReferenciaBancariaBO.cs (GAP-E — Bug crítico)	25
2. Impacto en modelos	29
Manejo de Errores y Excepciones	30
Estrategia de Pruebas	31
1. Pruebas funcionales (Criterios de Aceptación en DEV)	31
2. Pruebas técnicas	31
Unitarias	31
Pruebas de integración	32
3. Casos críticos	32



Importante

Posterior a este diseño, ¿cómo saber si el diseño de la solución al requisito está completo para que el programador inicie con el desarrollo?

Hazte estas preguntas rápidas:

- ¿El programador sabe qué flujo implementar?
- ¿Sabe qué pasa si algo falla?
- ¿Sabe qué reglas no puede romper?
- ¿Sabe qué pruebas debe pasar?
- ¿Sabe dónde impacta?

Nota: Este documento es una propuesta basada en el estándar IEEE 1016 “Software Design Description” que va permitir abarcar los puntos más importantes para el diseño del requisito que se está trabajando. Se debe administrar el tiempo que se tiene de diseño para que se complete de la mejor manera considerando todos los detalles técnicos.



Introducción

1. Propósito del documento

El propósito de este documento es definir el diseño de la solución técnica para el requisito **R16A-RE-FU-006 — Referencia de Pago y Código Validador**, describiendo la nueva tabla de BD, las clases de lógica de negocio, el controlador de API REST, la modificación a la fábrica de proformas y la estrategia de migración de datos necesarios para su implementación en el repositorio **ProquifaDotNet**.

Nota: Este documento se enfoca exclusivamente en el diseño de la solución en el **Back-end**. No redefine requisitos funcionales ni incluye diseño de pantallas o componentes Angular (scope de Front-end separado).

2. Alcance

Específicamente incluye:

- Creación de la tabla **dbo.ClienteDatosBancarios** (relación N:N cliente-cuenta bancaria con Código Validador y **ReferenciaVigente**).
- Creación de **ClienteDatosBancariosBO** — CRUD de la relación cliente-cuenta-CódigoValidador; calcula y persiste **ReferenciaVigente** en cada INSERT / UPDATE.
- Creación de **ReferenciaBancariaBO** — algoritmo de construcción de referencia bancaria (Banamex / no-Banamex); invocado desde **ClienteDatosBancariosBO**, no desde el factory.
- Creación de **ClienteDatosBancariosController** — API REST /**ClienteDatosBancarios**.
- Modificación de **tpProformaPedidoFactory** — leer **ClienteDatosBancarios.ReferenciaVigente** y asignarla a **ReferenciaPago** en lugar de **null**.
- Nuevos endpoints sobre **vEmpresaDatosBancarios** — catálogo de cuentas bancarias del grupo PROQUIFA para los selectores de la pantalla "Referencia de Pago".
- Creación de **vClienteDatosBancarios** — vista con relaciones completas de la asignación cliente-cuenta.
- Migración de datos Legacy: cuentas bancarias asignadas por cliente y sus Códigos Validadores desde PConnect (paquete SSIS — OBS-010).

No se consideran:

- Diseño de pantallas o componentes Angular de la sub-sección "Referencia de Pago" (scope de FE).
- Validación de formato o longitud del Código Validador — pendiente definir con cliente (DUDA-015).
- Clientes de Perú — modelo bancario peruano no definido; brecha abierta (BRECHA-03).



- Historial completo (bitácora) del Código Validador — se persiste solo el último cambio (OBS-014). La vista en pantalla del historial queda fuera del scope de R16 hasta confirmar con cliente (DUDA-120).
 - Restricción de rol para asignar cuentas — desestimada en versión inicial (DUDA-017 cerrada).
-



Visión general del diseño

1. Objetivo técnico

Implementar la persistencia de la relación cliente-cuenta bancaria con Código Validador en la nueva tabla **ClienteDatosBancarios**, calcular y persistir la referencia bancaria resultante en **ReferenciaVigente** al momento de asignar o actualizar la cuenta, exponer operaciones CRUD vía API REST, y modificar la fábrica de proformas para que **lea ReferenciaVigente** en lugar de recalcular la referencia, garantizando consistencia entre proformas y con el módulo Buzón de Pagos.

2. Componentes involucrados

Aplicativo	Componente	Responsabilidad	Ubicación
ProquifaDotNet	ClienteDatosBancariosController	Expone CRUD REST de asignaciones cliente-cuenta-CódigoValidador	<i>WebApi.Catalogos\Controllers\Configuracion\Clientes</i>
ProquifaDotNet	ClienteDatosBancariosBO	Lógica de persistencia: INSERT, UPDATE, DELETE lógico, GET; calcula ReferenciaVigente en cada escritura	<i>Logic.Pqf.Catalogos\Clientes\DatosBancarios</i>
ProquifaDotNet	ReferenciaBancariaBO	Algoritmo de construcción de referencia bancaria: Banamex (7 segmentos) y no-Banamex (nombre del cliente); invocado por ClienteDatosBancariosBO	<i>Logic.Pqf.Catalogos\Clientes\DatosBancarios</i>
ProquifaDotNet	tpProformaPedidoFactory	Fábrica de proforma — lee ClienteDatosBancarios.ReferenciaVigente y asigna a ReferenciaPago	<i>Logic.Pqf.Logistica\L05.TramitarPedido\Facturas\Fabrica</i>
ProquifaDotNet	EmpresaDatosBancariosBO / vEmpresaDatosBancarios	Catálogo de cuentas bancarias del grupo PROQUIFA; fuente para los selectores de banco y cuenta. Estructura real (verificada live 29-jun): EmpresaDatosBancarios es junction (IdEmpresaDatosBancarios , IdDatosBancarios , IdEmpresa ,	<i>Logic.Pqf.Catalogos\Cuentas</i>

Aplicativo	Componente	Responsabilidad	Ubicación
		Activo); el detalle de cuenta (NumeroDeCuenta , Beneficiario , Sucursal , Clabe , IdCatMoneda , NumeroTarjeta) vive en DatosBancarios ; el banco en catBanco (IdCatBanco , Banco , Clave); la moneda en catMoneda (ClaveMoneda , Moneda). La vista debe unir las 4 tablas	
ProquifaDotNet	vClienteDatosBancariosController	Vista de consulta con relaciones completas: datos bancarios (IdCatBanco , Banco , NumeroDeCuenta , Beneficiario , Sucursal , Clabe , IdCatMoneda , ClaveMoneda , Moneda , NumeroTarjeta) y cliente (Nombre , RS , ISORegion , Region)	<i>WebApi.Catalogos\Controllers\Configuracion\Cientes\Relaciones</i>
BD ProquifaDotNet	dbo.ClienteDatosBancarios	Tabla nueva — relación N:N Cliente ↔ EmpresaDatosBancarios con CodigoValidador , ReferenciaVigente e historial de último cambio	BD ProquifaDotNet
PConnect (Legacy)	CuentaCliente	Fuente de datos para la migración SSIS de RE-006 (asignaciones cliente-cuenta + CodValidador) — tabla en BD Legacy, solo lectura. CuentaBanco corresponde a la migración de EmpresaDatosBancarios (RE-FU-001), no a RE-006.	BD PConnect
SSIS	Paquete migración RE-006	Migra asignaciones cliente-cuenta + CódigoValidador de PConnect a ClienteDatosBancarios en PQF2	PConnect / SSIS





Diseño funcional detallado

1. Flujo 1 — Asignación de cuenta bancaria al cliente (CREATE / UPDATE)

Flujo iniciado desde Frontend cuando el usuario guarda una asignación nueva o modifica una existente en la sub-sección "Referencia de Pago" del Catálogo de Clientes.

1. El Frontend realiza **POST /ClienteDatosBancarios** (nueva asignación) o **PUT /ClienteDatosBancarios/Codigo** (modificación de Código Validador).
 Body POST:


```
{
  "IdCliente": "Guid",
  "IdEmpresaDatosBancarios": "Guid",
  "CodigoValidador": "string"
}
```

 Body PUT:


```
{
  "IdClienteDatosBancarios": "Guid",
  "CodigoValidador": "string"
}
```
2. **ClienteDatosBancariosController** recibe la solicitud y delega en **ClienteDatosBancariosBO**.
3. **Validaciones previas en el BO:**
 - Verificar que **EmpresaDatosBancarios.Activo = 1** para la cuenta seleccionada (RT-01).
 - Verificar que no exista ya una asignación activa con el mismo par (**IdCliente**, **IdEmpresaDatosBancarios**) — prevenir duplicados (RT-02).
 - Verificar que **CodigoValidador** no sea nulo ni vacío (CA-E1).
4. Si es **nueva asignación (POST)**:
 - Insertar registro en **ClienteDatosBancarios** con **IdCliente**, **IdEmpresaDatosBancarios**, **CodigoValidador**, **FechaRegistro = GETDATE()**, **IdUsuarioModificacion = ObtenerUsuarioAutenticado().IdUsuario** (usuario autenticado de la sesión — RT-09), **Activo = 1**.
 - Los campos de historial (**CodigoValidadorAnterior**, **FechaModificacionAnterior**, **IdUsuarioModificacionAnterior**) quedan en **NULL**.
 - Invocar **ReferenciaBancariaBO.Construir(cliente, empresaDatosBancarios, codigoValidador)** para calcular la referencia bancaria.
 - Persistir resultado en **ReferenciaVigente** y **FechaReferenciaVigente = GETDATE()**.
5. Si es **modificación de Código Validador (PUT /ClienteDatosBancarios/Codigo)**:
 - Leer el registro actual en **ClienteDatosBancarios**.
 - Copiar **CodigoValidador** vigente → **CodigoValidadorAnterior**.

- Copiar **FechaUltimaActualizacion** vigente → **FechaModificacionAnterior**.
 - Copiar **IdUsuarioModificacion** vigente → **IdUsuarioModificacionAnterior**.
 - Actualizar **CodigoValidador** con el nuevo valor, **FechaUltimaActualizacion** = **GETDATE()**, **IdUsuarioModificacion** = **ObtenerUsuarioAutenticado().IdUsuario** (usuario autenticado de la sesión — RT-09) (OBS-014).
 - Recalcular **ReferenciaBancariaBO.Construir()** con el nuevo **CodigoValidador**.
 - Actualizar **ReferenciaVigente** y **FechaReferenciaVigente** = **GETDATE()**.
6. El BO retorna el registro actualizado.
 7. **ClienteDatosBancariosController** responde **200 OK** con el registro; **409 Conflict** si la asignación ya existe (duplicado activo); o **400 Bad Request** con detalle si falla cualquier otra validación (cuenta inactiva, **CodigoValidador** vacío).

2. Flujo 2 — Asignación de referencia bancaria al generar proforma

OBS-013 actualizada: la referencia bancaria se calcula al configurar/actualizar la cuenta del cliente (Flujo 1) y se persiste en **ClienteDatosBancarios.ReferenciaVigente**. Al generar la proforma, el factory lee este campo y lo persiste en **tpProformaPedido.ReferenciaPago** como snapshot inmutable. No se recalcula en el momento de la generación — garantiza consistencia con el módulo Buzón de Pagos y elimina el riesgo de inconsistencia entre proformas re-emitidas.

1. El usuario tramita un pedido y el sistema inicia la generación de la proforma.
2. **tpProformaPedidoFactory** construye el objeto **tpProformaPedido**.
3. El factory —que ya recibe la **Empresa** emisora en su constructor (**tpProformaPedidoFactory(Empresa empresa)**)— consulta **ClienteDatosBancarios** unida a **EmpresaDatosBancarios**, filtrando por **IdCliente** del cliente de la proforma y **EmpresaDatosBancarios.IdEmpresa** = **empresa.IdEmpresa** (la entidad PROQUIFA que emite la proforma), y recupera la **ReferenciaVigente** de la asignación activa correspondiente. Esto resuelve **DUDA-118**: el cliente paga a la cuenta de la entidad que le factura (RT-08).
4. El factory asigna **tpProformaPedido.ReferenciaPago** = **ReferenciaVigente**. Si el cliente tiene **más de una** asignación activa para la misma empresa emisora (caso residual), se toma la más reciente (**FechaUltimaActualizacion DESC**) como desempate determinista. Si no existe asignación activa para esa empresa, **ReferenciaPago** = **null** — no bloquea la generación.
5. Al confirmar el envío exitoso del correo, el PDF se persiste con la referencia como snapshot inmutable — las proformas históricas conservan su referencia aunque cambien los datos del cliente o la cuenta.
6. Al consultar un PDF almacenado, el sistema sirve el archivo guardado sin regenerar.

Algoritmo Banamex — 7 segmentos (invocado en Flujo 1)

GAP-E cerrado: **dbo.Cliente** en PQF2 no tiene campo **Clave**. La clave numérica de Legacy se obtiene desde **PConnectProquifaDotNet.dbo.Clientes** (columna **ClienteLegacy int**; cruce por **ClientePQF** = **IdCliente**). El código actual de Juan David en **ReferenciaBancariaBO** usa

cliente.Clave — **bug crítico** que producirá S4 = "0000" para todos los clientes. Fix requerido: sustituir por consulta a **PConnectProquifaDotNet.dbo.Clientes**. Ver sección Impacto Técnico. (Verificado live 29-jun: *ProquifaDotNet.dbo.Carga_ClientesR1 DESCARTADA* como fuente — cobertura ~19% (268 *IdCliente* distintos vs 1,400 clientes PQF2) y filas duplicadas. *PConnectProquifaDotNet.dbo.Clientes* **verificada live 29-jun** vía cross-DB desde *proquifa-db-dev*: columnas *ClienteLegacy* *int* / *ClientePQF* *uniqueidentifier* confirmadas; cobertura **1,398 / 1,400 clientes PQF2 (99.9%)**. Los 2 clientes sin mapeo ("ACCESORIOS, SUMINISTROS Y EQUIPO MEDICO DE MEXICO" y "REHIDRATACION TOTAL R") producirán S4 = "0000" — fallback aceptado. La tabla contiene 4,173 filas por bug del job de sync (INSERT sin dedup): el lookup usa TOP 1 ORDER BY FechaRegistro ASC como tiebreaker. 6 *ClienteLegacy* mapean a 2 *ClientePQF* distintos cada uno; no afecta el lookup porque la query filtra por *ClientePQF* = @IdCliente.)

Segmento	Fuente en PQF2	Fallback
S1	Cliente.Nombre[0] — primera letra	'X' si nombre vacío o nulo
S2	Cliente.Nombre[1] — segunda letra	'X' si nombre tiene < 2 chars
S3	Cliente.Nombre[2] — tercera letra	'X' si nombre tiene < 3 chars
S4	Últimos 4 chars de CAST(ClienteLegacy AS varchar) desde PConnectProquifaDotNet.dbo.Clientes WHERE ClientePQF = @IdCliente	Padding '0' izq. si < 4 chars; "0000" si cliente no existe en tabla de mapeo
S5	catBanco.Clave — código del banco ('002' para Banamex)	—
S6	'P' si catMoneda.ClaveMoneda = 'MXN' ; 'D' en otro caso. Ruta real: DatosBancarios.IdCatMoneda → catMoneda.ClaveMoneda (la moneda NO está en EmpresaDatosBancarios)	—
S7	ClienteDatosBancarios.CodigoValidador	—

Ejemplo: Cliente **QUIMICOS SA DE CV**, **ClienteLegacy** = **12345**, Banamex (**002**), Moneda **MXN**, CodValidador **ABC** → referencia = **QUI2345002PABC**

GAP-D cerrado: identificación de Banamex confirmada en BD PQF2 — **catBanco.Clave** = **'002'** (validado live 29-jun: fila Banco='Banamex', Clave='002'). El cruce con tabla

Empresas descrito en P72 (DUDA-016) queda descartado; el campo **Clave** de **catBanco** es la condición definitiva.

Desbordamiento S4: registro actual en PConnect ~7,237 clientes. Al superar 9,999, **ClienteLegacy** con 5+ dígitos genera S4 repetido (ej. cliente 10,001 → S4 = "0001"). Riesgo aceptado: la unicidad de la referencia completa la aporta S7 (**CodigoValidador**). Si Banamex exige S4 único en el futuro, escalar a 5 chars requiere coordinación con el banco.

3. Criterios de aceptación del requisito

CA	Descripción	Estado	Justificación
CA-1	Usuario navega a Cobros del cliente y la sub-sección "Referencia de Pago" es accesible	Cubierto	POST /vClienteDatosBancarios con QueryInfo { filtros: { IdCliente } } retorna asignaciones activas del cliente con relaciones completas incluyendo ReferenciaVigente .
CA-2	Selector de Banco muestra catálogo de bancos del grupo PROQUIFA	Cubierto	Nuevo endpoint POST /vEmpresaDatosBancarios/Bancos — retorna bancos agrupados (IdCatBanco, Banco) desde vEmpresaDatosBancarios .
CA-3	Selector de Cuenta filtra por banco seleccionado	Cubierto	Nuevo endpoint POST /vEmpresaDatosBancarios con QueryInfo { filtros: { IdCatBanco } } — retorna cuentas del banco seleccionado; scope de FE el componente en cascada.
CA-4	Sucursal se autopobla desde la cuenta seleccionada (solo lectura)	Cubierto	Campo Sucursal incluido en respuesta de POST /vEmpresaDatosBancarios ; read-only en FE.
CA-5	CódigoValidador se acepta como input manual sin validación de formato/longitud	Cubierto	CodigoValidador varchar(50) sin constraint de formato; longitud provisional 50 — pendiente confirmar máximo con cliente (DUDA-015 abierta)
CA-6	Persistencia de nueva asignación o modificación — referencia	Cubierto	Flujo 1 pasos 4-5: referencia calculada por ReferenciaBancariaBO y persistida en ReferenciaVigente al guardar la

CA	Descripción	Estado	Justificación
	generada/actualizada a nivel cliente		asignación; Flujo 2: factory solo lee ese campo
CA-7	Cliente puede tener múltiples cuentas asignadas sin límite máximo definido	Cubierto	INSERT permite múltiples registros por cliente. La selección de cuenta al generar la proforma se resuelve por empresa emisora (RT-08, Flujo 2). El tope máximo de cuentas por cliente sigue sin definir con cliente (DUDA-118, no bloqueante)
CA-8	Eliminación lógica de asignación — Activo = 0 , no DELETE físico	Cubierto	DELETE /ClienteDatosBancarios?IdClienteDatosBancarios={guid} ejecuta UPDATE Activo = 0
CA-E1	CódigoValidador vacío → error, no guarda	Cubierto	Validación en ClienteDatosBancariosBO paso 3; responde 400 Bad Request
CA-E2	Cuentas con Activo = 0 no aparecen en selector	Cubierto	Filtro EmpresaDatosBancarios.Activo = 1 en endpoint POST /vEmpresaDatosBancarios.
CA-EC1	Asignación duplicada (misma cuenta al mismo cliente) bloqueada	Cubierto	Validación de duplicado en ClienteDatosBancariosBO paso 3; responde 409 Conflict
CA-EC2	Cuenta inactivada/eliminada post-asignación: proformas existentes intactas; no se generan nuevas con esa cuenta	Cubierto	Proformas existentes conservan su snapshot en PDF (OBS-013). Para proformas nuevas , una asignación cuya cuenta esté inactiva (EmpresaDatosBancarios.Activo = 0) no se considera activa → ReferenciaPago = null (RT-08). No se permite crear nuevas asignaciones sobre cuentas inactivas (RT-01). El sistema no altera asignaciones existentes de forma automática



CA	Descripción	Estado	Justificación
CA-12	Sin restricción de rol — cualquier usuario con acceso a cartera puede operar	Cubierto	Sin middleware de rol en controller (DUDA-017 cerrada)

4. Reglas técnicas aplicadas

Regla	Descripción
RT-01	ClienteDatosBancariosBO verifica EmpresaDatosBancarios.Activo = 1 antes de insertar cualquier asignación. No se permite asociar cuentas inactivas.
RT-02	La unicidad (IdCliente , IdEmpresaDatosBancarios) se garantiza con un índice UNIQUE filtrado (WHERE Activo = 1) . La baja lógica (Activo = 0) no cuenta como duplicado — permite reasignar una cuenta previamente eliminada al mismo cliente.
RT-03	El algoritmo Banamex en ReferenciaBancariaBO aplica la concatenación determinista de 7 segmentos. Los fallbacks ('X' en S1-S3, padding '0' en S4) se aplican siempre para garantizar formato consistente.
RT-04	tpProformaPedidoFactory lee ClienteDatosBancarios.ReferenciaVigente y la asigna a ReferenciaPago . No invoca ReferenciaBancariaBO — el cálculo ocurre en Flujo 1 (asignación), no en Flujo 2 (generación de proforma).
RT-05	Al modificar CodigoValidador : el valor e IdUsuarioModificacion vigentes se mueven a CodigoValidadorAnterior , FechaModificacionAnterior e IdUsuarioModificacionAnterior ; luego IdUsuarioModificacion se actualiza con el usuario actual. Se recalcula ReferenciaVigente con el nuevo código. Si CodigoValidadorAnterior ya tenía valor previo, se sobrescribe — no es bitácora completa, solo último cambio (OBS-014).
RT-06	Baja lógica (Activo = 0), nunca DELETE físico en ClienteDatosBancarios . Preserva integridad referencial y trazabilidad.
RT-07	La referencia bancaria se calcula en ClienteDatosBancariosBO al crear o actualizar la asignación, y se persiste en ClienteDatosBancarios.ReferenciaVigente . El factory de proformas lee este campo como fuente de verdad, garantizando que el Buzón de Pagos compare contra el mismo valor que apareció en el documento enviado al cliente.
RT-08	Al generar la proforma, tpProformaPedidoFactory selecciona la asignación de ClienteDatosBancarios cuya EmpresaDatosBancarios.IdEmpresa = empresa.IdEmpresa (empresa emisora de la proforma, ya inyectada en el

Regla	Descripción
	constructor del factory). Sólo se consideran asignaciones con ClienteDatosBancarios.Activo = 1 y EmpresaDatosBancarios.Activo = 1 . Si hay más de una para la misma empresa, se toma la más reciente (FechaUltimaActualizacion DESC). Si no hay ninguna, ReferenciaPago = null . Resuelve DUDA-118 y CA-EC2.
RT-09	El controller obtiene el usuario que ejecuta la operación mediante BaseApiController.ObtenerUsuarioAutenticado().IdUsuario (resuelto desde el token de autorización; patrón ya usado en ContratoClienteController) y lo pasa al BO para poblar IdUsuarioModificacion . En un UPDATE, el IdUsuarioModificacion vigente pasa a IdUsuarioModificacionAnterior y el usuario autenticado actual queda como IdUsuarioModificacion (par de FechaUltimaActualizacion).



Diseño de componentes

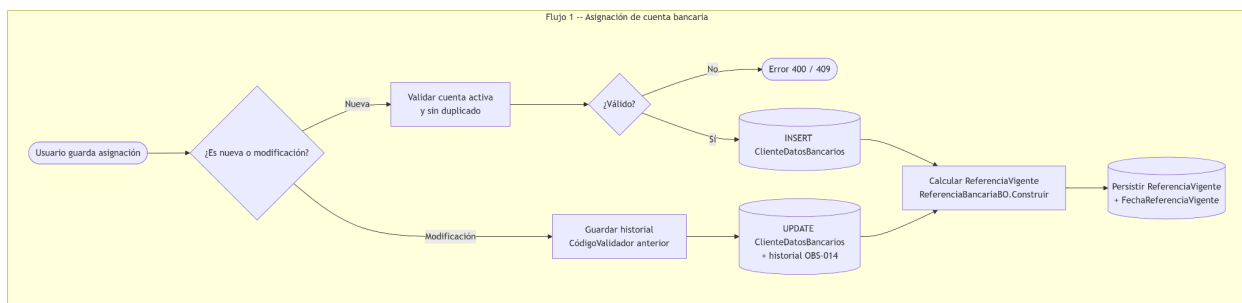
1. Diagramas

Flujo 1: **FE** → **ClienteDatosBancariosController** → **ClienteDatosBancariosBO** → **ReferenciaBancariaBO** → **dbo.ClienteDatosBancarios**

```

```mermaid
flowchart TD
 subgraph F1["Flujo 1 -- Asignación de cuenta bancaria"]
 A([Usuario guarda asignación]) --> B{¿Es nueva o modificación?}
 B -- Nueva --> C[Validar cuenta activa y sin duplicado]
 B -- Modificación --> D[Guardar historial y CódigoValidador anterior]
 C --> E{¿Válido?}
 E -- No --> F([Error 400 / 409])
 E -- Sí --> G[(INSERT ClienteDatosBancarios)]
 D --> G2[(UPDATE ClienteDatosBancarios + historial OBS-014)]
 G --> R[Calcular ReferenciaVigente y ReferenciaBancariaBO.Construir]
 G2 --> R
 R --> S[(Persistir ReferenciaVigente + FechaReferenciaVigente)]
 end

```



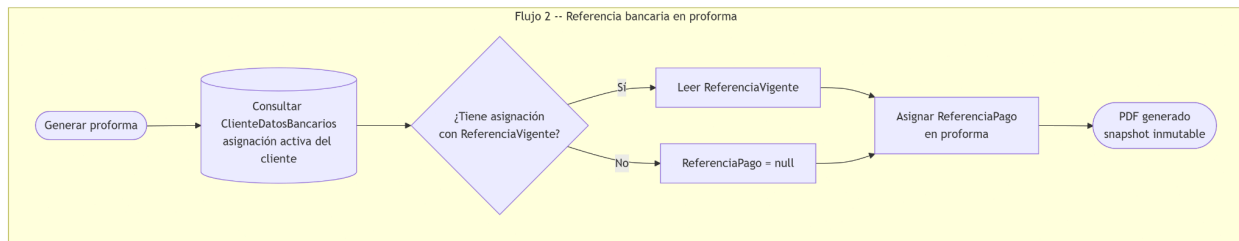


## Flujo 2: tpProformaPedidoFactory → dbo.ClienteDatosBancarios (lee ReferenciaVigente)

```

```mermaid
flowchart TD
    subgraph F2["Flujo 2 -- Referencia bancaria en proforma"]
        H([Generar proforma]) --> I([Consultar
        ClienteDatosBancarios\nasignación activa del cliente])
        I --> J{"¿Tiene asignación\ncon ReferenciaVigente?"}
        J -- No --> K[ReferenciaPago = null]
        J -- Sí --> L[Leer ReferenciaVigente]
        L --> P[Asignar ReferenciaPago\nen proforma]
        K --> P
        P --> Q([PDF generado\nsnapshot inmutable])
    end
    ```

```



# Diseño de Interfaces

## 1. Interfaces de entrada

Interfaz	Descripción
<b>POST /vClienteDatosBancarios</b>	Consulta asignaciones cliente-cuenta con relaciones completas. Body: <b>QueryInfo { filtros: { IdCliente } }</b> . Respuesta incluye datos bancarios y del cliente
<b>POST /ClienteDatosBancarios</b>	Crea nueva asignación cliente-cuenta. Body: { "IdCliente": "Guid", "IdEmpresaDatosBancarios": "Guid", "CodigoValidador": "string" }.
<b>PUT /ClienteDatosBancarios/Codigo .</b>	Actualiza <b>CodigoValidador</b> de una asignación existente y recalcula <b>ReferenciaVigente</b> . Body: { "IdClienteDatosBancarios": "Guid", "CodigoValidador": "string" }.
<b>DELETE /ClienteDatosBancarios?IdCliente DatosBancarios={guid} .</b>	Baja lógica de la asignación ( <b>Activo = 0</b> )
<b>POST /vEmpresaDatosBancarios/Bancos</b>	Retorna bancos del grupo PROQUIFA agrupados. Body: <b>QueryInfo { }</b> . Respuesta: <b>[{ IdCatBanco, Banco}]</b> — alimenta selector de Banco en FE
<b>POST /vEmpresaDatosBancarios</b>	Retorna cuentas filtradas por banco. Body: <b>QueryInfo { filtros: { IdCatBanco } }</b> . Respuesta incluye <b>Sucursal</b> para autopoblar campo read-only en FE



## 2. Interfaces de salida

Interfaz	Descripción
<b>tpProformaPedido.ReferenciaPago</b>	String con la referencia bancaria vigente leída de <b>ClienteDatosBancarios.ReferenciaVigente</b> ; asignado por <b>tpProformaPedidoFactory</b> al generar la proforma

## 3. Contrato de datos

Campos que expone **ClienteDatosBancarios** en las respuestas del controller:

```
{
 "IdClienteDatosBancarios": "Guid",
 "IdCliente": "Guid",
 "IdEmpresaDatosBancarios": "Guid",
 "CodigoValidador": "string",
 "ReferenciaVigente": "string",
 "Activo": "bool"
}
```

Campos de historial (solo BD, no expuestos en API en esta versión):

- **CodigoValidadorAnterior** (string, nullable)
- **FechaModificacionAnterior** (datetime, nullable)
- **IdUsuarioModificacionAnterior** (Guid, nullable)
- **IdUsuarioModificacion** (Guid, nullable) — usuario de la última modificación
- **FechaReferenciaVigente** (datetime, nullable)

## 4. Tabla de EndPoints

Solución / Proyecto	EndPoint	Parámetros	Salida
Catalogos	<b>POST /ClienteDatos Bancarios</b>	{ "IdCliente": "Guid", "IdEmpresaDatosBancarios": "Guid", "CodigoValidador": "string" }	<b>ClienteDatosBancarios</b>



Catalogos	<b>PUT</b> <b>/ClienteDatosBancarios/Codigo</b>	{ "IdClienteDatosBancarios": "Guid", "CodigoValidador": "string" }	<b>ClienteDatosBancarios</b>
Catalogos	<b>DELETE</b> <b>/ClienteDatosBancarios</b>	?IdClienteDatosBancarios={guid}	<b>204 No Content</b>
Catalogos	<b>POST</b> <b>/vClienteDatosBancarios</b>	QueryInfo { filtros: { IdCliente } }	<b>**List**</b>
Catalogos	<b>POST</b> <b>/vEmpresaDatosBancarios/Bancos</b>	QueryInfo {}	<b>[{ IdCatBanco, Banco }]</b>
Catalogos	<b>POST</b> <b>/vEmpresaDatosBancarios</b>	QueryInfo { filtros: { IdCatBanco } }	<b>**List**</b>



# Diseño de Modelo de Datos

## 1. Nueva tabla — dbo.ClienteDatosBancarios

```
CREATE TABLE dbo.ClienteDatosBancarios
(
 IdClienteDatosBancarios uniqueidentifier NOT NULL
 CONSTRAINT PK_ClienteDatosBancarios PRIMARY KEY
 CONSTRAINT DF_ClienteDatosBancarios_Id DEFAULT (NEWSEQUENTIALID()),

 IdCliente uniqueidentifier NOT NULL
 CONSTRAINT FK_ClienteDatosBancarios_Cliente
 FOREIGN KEY REFERENCES dbo.Cliente(IdCliente),

 IdEmpresaDatosBancarios uniqueidentifier NOT NULL
 CONSTRAINT FK_ClienteDatosBancarios_EmpresaDatosBancarios
 FOREIGN KEY REFERENCES
dbo.EmpresaDatosBancarios(IdEmpresaDatosBancarios),

 -- Longitud provisional varchar(50); confirmar máximo con cliente
 (DUDA-015 abierta)
 CodigoValidador varchar(50) NULL,

 -- Referencia bancaria calculada al asignar/actualizar CodigoValidador
 (RT-07)
 ReferenciaVigente varchar(200) NULL,
 FechaReferenciaVigente datetime NULL,

 -- Historial último cambio (OBS-014, sesión 10-JUN-2026) -- GAP-B
 confirmado
 CodigoValidadorAnterior varchar(50) NULL,
 FechaModificacionAnterior datetime NULL,
 IdUsuarioModificacionAnterior uniqueidentifier NULL,

 FechaRegistro datetime NOT NULL
 CONSTRAINT DF_ClienteDatosBancarios_FechaRegistro DEFAULT
(GETDATE()),
 FechaUltimaActualizacion datetime NOT NULL
 CONSTRAINT DF_ClienteDatosBancarios_FechaActualizacion DEFAULT
(GETDATE()),
 -- Usuario de la última modificación (par de FechaUltimaActualizacion)
 IdUsuarioModificacion uniqueidentifier NULL,
```



```

 Activo bit NOT NULL
 CONSTRAINT DF_ClienteDatosBancarios_Activo DEFAULT (1)
);

-- Unicidad solo entre asignaciones ACTIVAS: un cliente no puede tener la
-- misma cuenta activa dos veces. La baja lógica (Activo = 0) libera el par
-- y permite re-asignar la cuenta al mismo cliente (RT-02). Un CONSTRAINT
-- UNIQUE inline no admite WHERE en SQL Server, por eso se usa índice
-- filtrado.
CREATE UNIQUE INDEX UQ_ClienteDatosBancarios_ClienteCuenta
 ON dbo.ClienteDatosBancarios (IdCliente, IdEmpresaDatosBancarios)
 WHERE Activo = 1;

```

## 2. Modificaciones en BD existente

No se modifican tablas existentes. Las tablas **EmpresaDatosBancarios**, **DatosBancarios**, **catBanco** y **catMoneda** se usan en modo lectura (la vista **vClienteDatosBancarios** las une).

### Flujo A — Migración inicial Legacy → PQF2 (carga única de arranque)

Extrae cuentas bancarias asignadas por cliente y sus Códigos Validadores desde **PConnect** y los carga en **ClienteDatosBancarios** en **ProquifaDotNet**. Sin esta carga, la pantalla "Referencia de Pago" aparecería vacía para todos los clientes existentes. Pendiente crear tarea formal (MIG-DATOS, est. 24 hrs).

Campo PConnect (CuentaCliente)	Campo PQF2 ClienteDatosBancarios	Notas
claveCliente (int)	IdCliente (uniqueidentifier)	Mapeo vía <b>PConnectProquifaDotNet.dbo.Clientes:</b> <b>WHERE ClienteLegacy = claveCliente</b> retorna <b>ClientePQF</b> (GUID de PQF2). GAP-E cerrado
idCuenta	IdEmpresaDatosBancarios	Mapeo por <b>CLABE</b> (verificado live 1-jul): <b>CuentaCliente.idCuenta</b> → <b>CuentaBanco.Clabe</b> → <b>DatosBancarios.Clabe</b> → <b>EmpresaDatosBancarios.IdEmpresaDatosBancarios</b> . Las 4 cuentas realmente usadas por las 7,075 asignaciones resuelven a



Campo PConnect (CuentaCliente)	Campo PQF2 ClienteDatosBancarios	Notas
		<b>exactamente un</b> IdEmpresaDatosBancarios (0 sin match, 0 ambiguo). <b>Caveats SSIS:</b> (a) la collation difiere (Legacy <code>SQL_Latin1_General_CP1_CI_AS</code> vs PQF2 <code>Modern_Spanish_CI_AS</code> ) → aplicar <code>COLLATE DATABASE_DEFAULT</code> en el join por Clabe; (b) <code>CuentaBanco</code> tiene 2 CLABEs duplicadas a nivel global (fuera de las 4 en uso) → si una Clabe resuelve a 0 o >1 cuentas PQF2, registrar error de fila y omitir sin cancelar el paquete.
<b>CodValidador</b>	<b>CodigoValidador</b>	Nombre real confirmado en PConnect: <b>CodValidador</b> . <b>CodValidar</b> en P72 S7 es typo. GAP-C cerrado.

Post-carga: calcular **ReferenciaVigente** para cada registro migrado invocando **ReferenciaBancariaBO**.

### Flujo B — Sincronización ongoing PQF2 → Legacy (paquete SSIS existente)

Unidireccional: cambios en **ClienteDatosBancarios** PQF2 (cuentas asignadas + **CodigoValidador**) se transfieren a Legacy. Legacy **no** sobrescribe PQF2. Solo aplica a clientes México — Perú excluido. El paquete SSIS existente en PConnect se extiende (no se reemplaza); puede requerir **ALTER TABLE** en tablas Legacy destino. Manejo de errores por fila obligatorio sin cancelar el paquete completo.



## Impacto Técnico

### 1. Impacto en código existente

#### Repositorio ProquifaDotNet

#	Archivo	Tipo de cambio
1	<b>BD: dbo.ClienteDatosBancarios</b>	Crear tabla nueva (DDL arriba)
2	<b>Logic.Pqf.Catalogos\Clientes\Datos Bancarios\ClienteDatosBancariosBO.cs</b>	Clase nueva — CRUD de relación cliente-cuenta-CódigoValidador; invoca <b>ReferenciaBancariaBO</b> y persiste <b>ReferenciaVigente</b> en cada escritura
3	<b>Logic.Pqf.Catalogos\Clientes\Datos Bancarios\ReferenciaBancariaBO.cs</b>	Clase nueva — algoritmo de construcción de referencia bancaria; invocada desde <b>ClienteDatosBancariosBO</b> , no desde el factory
4	<b>Logic.Pqf.Logistica\L05.TramitarPedido\Facturas\Fabrica\tpProformaPedidoFactory.cs</b>	Modificar — leer <b>ClienteDatosBancarios.ReferenciaVigente</b> filtrando por <b>IdCliente + empresa.IdEmpresa</b> (ya disponible en el constructor <b>tpProformaPedidoFactory(Empresa empresa)</b> ) y asignar a <b>ReferenciaPago</b> ; eliminar cualquier llamada directa a <b>ReferenciaBancariaBO</b> (RT-08)
5	<b>WebApi.Catalogos\Controllers\Configuracion\Clientes\ClienteDatosBancariosController.cs</b>	Controller nuevo — <b>POST, PUT /Codigo</b> (id en body), <b>DELETE</b> (?IdClienteDatosBancarios querystring). Hereda de <b>BaseApiController</b> y obtiene el usuario vía <b>ObtenerUsuarioAutenticado().IdUsuario</b> para poblar <b>IdUsuarioModificacion</b> (RT-09)
6	Nuevos endpoints <b>vEmpresaDatosBancarios</b>	Nuevo — <b>POST /vEmpresaDatosBancarios/Bancos</b> y <b>POST /vEmpresaDatosBancarios</b> con QueryInfo
6b	<b>BD: dbo.spObtenerClienteLegacyId</b>	SP nuevo — recibe <b>@IdCliente uniqueidentifier</b> , retorna <b>int?</b> con <b>ClienteLegacy</b> desde <b>PConnectProquifaDotNet.dbo.Clientes</b>



#	Archivo	Tipo de cambio
		(TOP 1 ORDER BY FechaRegistro ASC). Patrón del ecosistema: el C# nunca instancia <b>PConnectProquiifaDotNetEntities</b> directamente en BOs.
7	Paquete SSIS (Flujo A) — análisis carga inicial Legacy → PQF2	Nuevo — mapear <b>CuentaCliente</b> PConnect → <b>ClienteDatosBancarios</b> PQF2: cliente vía <b>PConnectProquiifaDotNet.dbo.Cientes</b> , cuenta vía <b>CLABE</b> ( <b>CuentaBanco.Clabe</b> → <b>DatosBancarios.Clabe</b> → <b>EmpresaDatosBancarios</b> , con <b>COLLATE</b> ); pendiente crear tarea formal MIG-DATOS
8	Paquete SSIS (Flujo A) — ejecución carga inicial	Nuevo — INSERT masivo con validaciones de integridad + cálculo de <b>ReferenciaVigente</b> por cada registro migrado
9	Paquete SSIS (Flujo A) — pruebas carga inicial	Nuevo — validar conteos y muestreo de registros migrados
10	Paquete SSIS existente (Flujo B, T6) — análisis campos R16	Extender — analizar campos <b>ClienteDatosBancarios</b> a mapear en el paquete SSIS existente en PConnect
11	Paquete SSIS existente (Flujo B, T7) — actualización .dtsx	Extender — agregar campos RE-006 ( <b>CodigoValidador</b> , cuentas asignadas) al paquete; puede requerir <b>ALTER TABLE</b> en Legacy
12	Paquete SSIS existente (Flujo B, T8) — pruebas sincronización	Nuevo — validar que cambios en PQF2 se transfieren correctamente a Legacy; verificar que Legacy no sobrescribe PQF2

#### Revisar adicionalmente:

- **WebApi.Catalogos\Controllers\Configuracion\Clientes\Relaciones\vClienteDatosBancariosController.cs** — actualizar vista para incluir relaciones completas: datos bancarios (**IdCatBanco**, **Banco**, **NumeroDeCuenta**, **Beneficiario**, **Sucursal**, **Clabe**, **IdCatMoneda**, **ClaveMoneda**, **Moneda**, **NumeroTarjeta**) y cliente (**Nombre**, **RS**, **ISORegion**, **Region**).

 Corrección requerida en **ReferenciaBancariaBO.cs** (GAP-E — Bug crítico)



El código actual de Juan David en **ReferenciaBancariaBO.ConstruirBanamex()** usa **cliente.Clave** para construir S4. La entidad **dbo.Cliente** en PQF2 **no tiene ese campo** — el código no compilará o producirá S4 = **"0000"** para todos los clientes.

#### Datos de BD confirmados live (29-jun-2026):

Dato	Valor
Tabla de mapeo	<b>PConnectProquifaDotNet.dbo.Clientes</b>
Columnas relevantes	<b>ClienteLegacy</b> (int, nullable) / <b>ClientePQF</b> (uniqueidentifier, nullable)
Total filas	4,173 — duplicados idénticos por bug del job de sync (INSERT sin dedup)
Cobertura vs PQF2	<b>1,398 / 1,400 clientes (99.9%)</b>
Clientes sin mapeo	2 — producen S4 = "0000" (fallback aceptado)
ClienteLegacy máximo	7,237 — S4 seguro hasta 9,999
Ambigüedad	6 <b>ClienteLegacy</b> mapean a 2 <b>ClientePQF</b> distintos cada uno — no afecta el lookup porque la query filtra <b>WHERE ClientePQF = @IdCliente</b>

**Fix recomendado — patrón parámetro (Juan David):** **ClienteDatosBancariosBO** resuelve **ClienteLegacy** antes de invocar el algoritmo, evitando el contexto EF adicional dentro de **ReferenciaBancariaBO**:

#### Paso 1 — modificar firma de **ReferenciaBancariaBO.Construir()**

```
// ANTES
public string Construir(Cliente cliente, EmpresaDatosBancarios cuenta,
string codigoValidador)

// DESPUÉS
public string Construir(Cliente cliente, int? clienteLegacy,
EmpresaDatosBancarios cuenta, string codigoValidador)
```

#### Paso 2 — reemplazar **cliente.Clave** en **ConstruirBanamex()**



```
// ANTES -- bug: cliente.Clave no existe en dbo.Cliente de PQF2
var clave = cliente.Clave ?? string.Empty;

// DESPUÉS -- recibe clienteLegacy como parámetro ya resuelto por
// ClienteDatosBancariosBO
var claveStr = clienteLegacy?.ToString() ?? string.Empty;
var seg4 = claveStr.Length >= 4
 ? claveStr.Substring(claveStr.Length - 4)
 : claveStr.PadLeft(4, '0');
```

### Paso 3 — resolver ClienteLegacy en ClienteDatosBancariosBO antes de invocar el algoritmo

Patrón del ecosistema: el C# nunca instancia **PConnectProquifaDotNetEntities** directamente en BOs. Los cruces cross-DB se encapsulan en SPs invocados desde **ProquifaDotNetEntities**. Ejemplos en producción: `spActualizarClienteLegacy` (ClienteBO), `etlClienteContactoProcesarLegacy` (ContactoBO), `etlClienteDireccionProcesarLegacy` (DireccionClienteBO).

```
// SP invocado desde ProquifaDotNetEntities -- patrón del ecosistema (NO EF
// directo a PConnectProquifaDotNetEntities)
// EL SP cruza internamente a PConnectProquifaDotNet.dbo.Clientes con TOP 1
// ORDER BY FechaRegistro ASC
// null si el cliente no tiene mapeo legacy → seg4 = "0000" (fallback
// aceptado, DIS RT-03)
int? clienteLegacy = null;
using (var db = new ProquifaDotNetEntities())
{
 clienteLegacy = db.spObtenerClienteLegacyId(cliente.IdCliente)
 .SingleOrDefault();
}

// Registrar advertencia si el cliente no tiene mapeo legacy
if (clienteLegacy == null)
 Logger.WarnFormat("ClienteLegacy no encontrado para IdCliente {0} -- S4
 será '0000'", cliente.IdCliente);

var referencia = _referenciaBancariaBO.Construir(
 cliente, clienteLegacy, empresaDatosBancarios, codigoValidador);
```





### Casos de prueba requeridos para ReferenciaBancariaBO (unitarias):

Caso	clienteLegacy	S4 esperado
Clave larga	12345	"2345"
Clave exacta 4 dígitos	1234	"1234"
Clave corta (2 dígitos)	12	"0012"
Clave 1 dígito	5	"0005"
Sin mapeo legacy	null	"0000"

## 2. Impacto en modelos

- **Nueva entidad: ClienteDatosBancarios** con todos sus campos (ver DDL), incluyendo **ReferenciaVigente** y **FechaReferenciaVigente**.
- **Entidad modificada: tpProformaPedido** — el campo **ReferenciaPago** ya existe y actualmente se asigna **null**; se asigna el string leído desde **ClienteDatosBancarios.ReferenciaVigente**.



## Manejo de Errores y Excepciones

Escenario	Comportamiento esperado
<b>POST /ClienteDatosBancarios</b> con cuenta <b>Activo = 0</b>	<b>ClienteDatosBancariosBO</b> rechaza; controller responde <b>400 Bad Request</b> con mensaje "La cuenta bancaria seleccionada no está activa."
<b>POST /ClienteDatosBancarios</b> con par ( <b>IdCliente</b> , <b>IdEmpresaDatosBancarios</b> ) ya existente y activo	<b>ClienteDatosBancariosBO</b> detecta duplicado; controller responde <b>409 Conflict</b> con mensaje "Esta cuenta ya está asignada al cliente."
<b>POST</b> o <b>PUT /Codigo</b> con <b>CodigoValidador</b> nulo o vacío	<b>ClienteDatosBancariosBO</b> rechaza; controller responde <b>400 Bad Request</b> con mensaje "El Código Validador es requerido."
<b>tpProformaPedidoFactory</b> — cliente sin asignación activa para la empresa emisora	<b>ReferenciaVigente</b> no encontrada; <b>ReferenciaPago</b> queda nulo en la proforma — no bloquea la generación del documento.
<b>ReferenciaBancariaBO</b> — nombre de cliente vacío o nulo (segmentos S1-S3)	Aplica fallback ' <b>X</b> ' por segmento faltante; no lanza excepción.
<b>ReferenciaBancariaBO</b> — <b>catBanco.Clave</b> no encontrado	Tratar como no-Banamex: <b>referencia = Cliente.Nombre</b> . No lanzar excepción. Registrar log de advertencia con <b>IdCatBanco</b> para diagnóstico.
Error al calcular <b>ReferenciaVigente</b> en Flujo 1	Propagar excepción; no persistir el registro con <b>ReferenciaVigente</b> nula salvo que sea el fallback esperado por falta de asignación Banamex.



# Estrategia de Pruebas

## 1. Pruebas funcionales (Criterios de Aceptación en DEV)

- Crear asignación nueva → registro creado en **ClienteDatosBancarios** con **ReferenciaVigente** calculada correctamente.
- Intentar crear asignación con cuenta inactiva → **400 Bad Request**.
- Intentar crear asignación duplicada (misma cuenta al mismo cliente) → **409 Conflict**.
- Intentar crear asignación sin **CodigoValidador** → **400 Bad Request**.
- Modificar **CodigoValidador** → valor anterior se mueve a **CodigoValidadorAnterior** con fecha y usuario; **ReferenciaVigente** se recalcula.
- Eliminar asignación → **Activo = 0**; no aparece en **POST /vClienteDatosBancarios** con filtro activo.
- Generar proforma para cliente con cuenta Banamex y **CodigoValidador ABC** → **ReferenciaPago** contiene la referencia de 7 segmentos almacenada en **ReferenciaVigente**.
- Generar proforma para cliente con cuenta no-Banamex → **ReferenciaPago** contiene el nombre del cliente almacenado en **ReferenciaVigente**.
- Generar proforma para cliente sin asignación activa → proforma se genera sin bloqueo; **ReferenciaPago** es nulo.
- Generar proforma para cliente con asignaciones en **dos empresas distintas** → **ReferenciaPago** corresponde a la cuenta de la **empresa emisora** de la proforma (RT-08), no a la de otra empresa.

## 2. Pruebas técnicas

### Unitarias

- **ReferenciaBancariaBO.Construir()** retorna **Cliente.Nombre** para banco con **catBanco.Clave ≠ '002'**.
- **ReferenciaBancariaBO.ConstruirBanamex()** retorna concatenación correcta de 7 segmentos con datos completos.
- **ReferenciaBancariaBO.ConstruirBanamex()** aplica fallback **'X'** en S1 cuando **Cliente.Nombre** está vacío.
- **ReferenciaBancariaBO.ConstruirBanamex()** aplica padding **'0'** en S4 cuando **PConnectProquifaDotNet.dbo.Clientes.ClienteLegacy** tiene < 4 dígitos.
- **ReferenciaBancariaBO.ConstruirBanamex()** retorna **'D'** en S6 cuando **catMoneda.ClaveMoneda ≠ 'MXN'** (resuelta vía **DatosBancarios.IdCatMoneda** → **catMoneda**).
- **ClienteDatosBancariosBO** rechaza inserción cuando **EmpresaDatosBancarios.Activo = 0**.
- **ClienteDatosBancariosBO** rechaza inserción cuando ya existe registro activo con mismo (**IdCliente**, **IdEmpresaDatosBancarios**).
- **ClienteDatosBancariosBO** mueve **CodigoValidador** a **CodigoValidadorAnterior** y recalcula **ReferenciaVigente** en UPDATE correctamente.



### Pruebas de integración

- **POST /ClienteDatosBancarios** → registro persiste en BD con todos los campos correctos incluyendo **ReferenciaVigente**.
- **PUT /ClienteDatosBancarios/Codigo** con body { **IdClienteDatosBancarios**, **CodigoValidador** } → campos de historial actualizados y **ReferenciaVigente** recalculada en BD.
- **DELETE /ClienteDatosBancarios?IdClienteDatosBancarios={guid}** → registro queda con **Activo = 0**, no eliminado físicamente.
- **tpProformaPedidoFactory** lee **ReferenciaVigente** y **tpProformaPedido.ReferenciaPago** tiene valor no nulo para cliente con asignación activa.
- Migración SSIS: conteo de registros en **ClienteDatosBancarios** post-migración iguala conteo en **CuentaCliente** de PConnect con **Activo = 1**; todos los registros tienen **ReferenciaVigente** no nula.

### 3. Casos críticos

- **Cliente con múltiples cuentas:** cliente con varias asignaciones activas → cada una conserva su propia **ReferenciaVigente**; al generar la proforma el factory elige la de la **empresa emisora** (RT-08). Si hay varias para la misma empresa, gana la más reciente (**FechaUltimaActualizacion DESC**). (*DUDA-118: mecanismo cerrado por diseño; resta confirmar con cliente sólo si existe un tope máximo de cuentas por cliente — no bloqueante.*)
- **Nombre de cliente muy corto:** cliente con nombre de 1 o 2 caracteres → S2 y/o S3 usan fallback 'X'; referencia Banamex se genera sin error.
- **CódigoValidador modificado tras generar PDF:** la proforma ya generada conserva la referencia original (snapshot en PDF); **ReferenciaVigente** se actualiza para proformas futuras.
- **Cuenta inactivada/eliminada tras asignación:** la asignación permanece en BD sin alteración automática; las proformas ya generadas conservan su snapshot. Para proformas nuevas, la cuenta inactiva (**EmpresaDatosBancarios.Activo = 0**) no se considera → **ReferenciaPago = null** (RT-08). No se permiten nuevas asignaciones sobre cuentas inactivas (RT-01). (*CA-EC2 cerrado: el sistema no toca proformas existentes y sólo impide crear nuevas referencias sobre cuentas inactivas.*)





## Control de versiones

Versión	Fecha	Autor	Tipo de Cambio	Descripción del cambio	Aprobó
1.0	19 jun 2026	Jose Armando...	Creación	Se documenta el diseño de la solución backend de RE-FU-006.	—
1.1	1 jul 2026	Jose Armando...	Revisión	Aplicación de comentarios de revisión (Juan David García Cruz, 24-jun-2026)	—

